

1)Virtual memory address

```
[medikondarakesh@Medikondas-MacBook-Air ~ % nano r.c
medikondarakesh@Medikondas-MacBook-Air ~ % gcc r.c -o r
```

```
[medikondarakesh@Medikondas-MacBook-Air ~ % ./r
Value of x = 10
Virtual address of x = 0x16f9c3198
```

```
#include <stdio.h>
int main() {
    int x = 10;
    printf("Value of x = %d\n",
    x);
    printf("Virtual address of x = %p\n", (void *)&x);
    return
    0;
}
```

```
⌘G Get Help      ⌘O WriteOut      ⌘R Read File     ⌘Y Prev Pg      ⌘K Cut Text      ⌘G Cur Pos
⌘X Exit          ⌘J Justify       ⌘W Where is     ⌘V Next Pg      ⌘U UnCut Text   ⌘T To Spell
```

2)process spaces

```
[medikondarakesh@Medikondas-MacBook-Air ~ % nano e.c
[medikondarakesh@Medikondas-MacBook-Air ~ % gcc e.c -o e
[medikondarakesh@Medikondas-MacBook-Air ~ % ./e
Code address      : 0x104334548
Data address      : 0x10433c000
BSS address       : 0x10433c004
Heap address      : 0x1049b18d0
Stack address     : 0x16bacb1a8
medikondarakesh@Medikondas-MacBook-Air ~ % █
```

```
UW PICO 5.09 File: e.c

#include <stdio.h>
#include <stdlib.h>

int global = 10;
int bss_variable;

int main() {

    int local = 20;

    int *heap = malloc(sizeof(int));
    *heap = 30;

    printf("Code address : %p\n", (void *)main);
    printf("Data address : %p\n", (void *)&global);
    printf("BSS address : %p\n", (void *)&bss_variable);
    printf("Heap address : %p\n", (void *)heap);
    printf("Stack address : %p\n", (void *)&local);

    free(heap);

    return 0;
}
```

⌘ Get Help ⌘ WriteOut ⌘ Read File ⌘ Prev Pg ⌘ Cut Text ⌘ Cur Pos
⌘ Exit ⌘ Justify ⌘ Where is ⌘ Next Pg ⌘ UnCut Text ⌘ To Spell

3)demand paging

```
heap address : 0x104701000
Stack address : 0x16bacb1a8
medikondarakesh@Medikondas-MacBook-Air ~ % nano p.c
medikondarakesh@Medikondas-MacBook-Air ~ % gcc p.c -o p
medikondarakesh@Medikondas-MacBook-Air ~ % ./p
Enter page number (0-3):
```

```
UW PICO 5.09 File: p.c

#include <stdio.h>

int main() {

    int page_table[4] = {1, -1, 3, -1};

    int page;

    printf("Enter page number (0-3): ");
    scanf("%d", &page);

    if (page_table[page] == -1) {
        printf("Page Fault!\n");
        printf("Page must be loaded into memory.\n");
    }
    else {
        printf("Page is already in memory.\n");
        printf("Physical frame = %d\n", page_table[page]);
    }

    return 0;
}
```

⌘ Get Help ⌘ WriteOut ⌘ Read File ⌘ Prev Pg ⌘ Cut Text ⌘ Cur Pos
⌘ Exit ⌘ Justify ⌘ Where is ⌘ Next Pg ⌘ UnCut Text ⌘ To Spell

4)Address Translation (Virtual to Physical Address)

```
[medikondarakesh@Medikondas-MacBook-Air ~ % nano t.c
[medikondarakesh@Medikondas-MacBook-Air ~ % gcc t.c -o t
[medikondarakesh@Medikondas-MacBook-Air ~ % ./t
Virtual Address = 250
Page Number      = 2
Offset           = 50
Frame Number     = 1
Physical Address = 150
medikondarakesh@Medikondas-MacBook-Air ~ %
```

```
UW PICO 5.09 File: t.c
#include <stdio.h>

int main() {

    int page_table[] = {3, 5, 1, 7};
    int page_size = 100;

    int virtual_address = 250;

    int page = virtual_address / page_size;
    int offset = virtual_address % page_size;

    int frame = page_table[page];

    int physical_address = frame * page_size + offset;

    printf("Virtual Address = %d\n", virtual_address);
    printf("Page Number      = %d\n", page);
    printf("Offset           = %d\n", offset);
    printf("Frame Number     = %d\n", frame);
    printf("Physical Address = %d\n", physical_address);

    return 0;
}
```

⌘ Get Help

⌘ Exit

⌘ WriteOut

⌘ Justify

⌘ Read File

⌘ Where is

⌘ Prev Pg

⌘ Next Pg

⌘ Cut Text

⌘ UnCut Text

⌘ Cur Pos

⌘ To Spell

5)Dynamic Memory Allocation

```
medikondarakesh@Medikondas-MacBook-Air ~ % nano w.c
medikondarakesh@Medikondas-MacBook-Air ~ % gcc w.c -o w
medikondarakesh@Medikondas-MacBook-Air ~ % ./w
10 20 30 40 50
medikondarakesh@Medikondas-MacBook-Air ~ % █
```

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p;

    p = malloc(5 * sizeof(int));

    if (p == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        p[i] = (i + 1) * 10;
    }

    for (int i = 0; i < 5; i++) {
        printf("%d ", p[i]);
    }

    printf("\n");

    free(p);

    return 0;
}
```

Get Help Exit WriteOut Justify Read File Where Is Prev Pg Next Pg Cut Text UnCut Text Cur Pos To Spell

6)Demonstrating Segmentation Fault

```
medikondarakesh@Medikondas-MacBook-Air ~ % nano q.c
medikondarakesh@Medikondas-MacBook-Air ~ % gcc q.c -o q
medikondarakesh@Medikondas-MacBook-Air ~ % ./q
Before accessing memory
zsh: segmentation fault ./q
medikondarakesh@Medikondas-MacBook-Air ~ % █
```

```
#include <stdio.h>

int main() {
    int *p = NULL;

    printf("Before accessing memory\n");

    *p = 100;

    printf("After accessing memory\n");

    return 0;
}
```